

iRDRC: An Intelligent Real-Time Dual-Functional Radar-Communication System for Automotive Vehicles

Detailed Section-by-Section Explanation (Sections I–IV)

Base paper: Hieu, Hoang, Luong, Niyato (IEEE WCL, Vol. 9, No. 12, Dec. 2020)
Explanatory notes prepared for BTP literature study

Abstract

This document is a detailed, section-by-section explanatory walkthrough of the base paper “iRDRC: An Intelligent Real-Time Dual-Functional Radar-Communication System for Automotive Vehicles” (DOI: 10.1109/LWC.2020.3014972), covering Sections I–IV (Introduction, System Model, Problem Formulation, and the Deep Reinforcement Learning Algorithm). Performance evaluation (Section V) and the conclusion are intentionally excluded, to be studied separately before finalizing the novelty direction for our own BTP work. Boxed **Remarks** throughout this document mark places where we go beyond what the paper explicitly states – either by pointing out simplifying assumptions the paper makes silently, or by connecting a detail to a potential research gap. These remarks are not part of the original paper; they are our own annotations.

Contents

1	Introduction	1
1.1	Motivating Context	1
1.2	What DFRC Does	1
1.3	The Core Problem	2
1.4	Prior Approaches and Their Limitations	2
1.5	The Paper’s Stated Contribution	2
2	System Model	3
2.1	Operating Principle (Introductory Paragraph)	3
2.2	II-A: Dual-Functional Radar-Communication Model	3
2.3	II-B: Environment Model	4
3	Problem Formulation	5
3.1	The MDP Tuple	5
3.2	III-A: Action Space and State Space	5
3.3	III-B: Reward Function	6
4	Deep Reinforcement Learning Algorithm	7
4.1	The Q-Network’s Role	7
4.2	The Training Loop	7
4.3	The Weight Update Equation	8
4.4	Convergence Criterion	8

1 Introduction

1.1 Motivating Context

The paper opens by establishing that autonomous vehicles (AVs) must navigate efficiently and safely in complex, uncontrolled environments. Dual-Functional Radar-Communication (DFRC) system design is introduced as a promising technology to help meet this requirement. DFRC itself is *not* this paper’s invention – it is an established research area that the paper builds directly upon.

1.2 What DFRC Does

DFRC lets a single piece of AV hardware perform two distinct functions:

- **Radar function:** detects distant objects or unexpected events, and critically, continues to work under poor visibility conditions (e.g. bad weather) – an advantage over purely camera-based sensing.
- **Communication function:** uses Vehicle-to-Infrastructure (V2I) links to transmit data (e.g. current traffic information, live onboard video) to Base Stations (BSs) distributed along the road, supporting intelligent road management, route selection, and data analysis.

1.3 The Core Problem

Because both functions share the *same physical hardware*, they must share system resources: antennas, spectrum, and power. The AV cannot run both functions at full capacity simultaneously. This creates the central question the paper answers: *at any given moment, should the AV operate in radar mode or communication mode?*

1.4 Prior Approaches and Their Limitations

Three prior resource-sharing approaches are reviewed:

- [2] adopts the IEEE 802.11ad standard: fixed “preamble blocks” in the frame are reserved for radar (range/velocity estimation), and “data blocks” for communication.
- [4] uses a time-sharing approach: fixed time cycles are split between radar mode and communication mode (rather than frame-based reservation).
- [3] focuses on reducing beam alignment overhead between AVs and infrastructure for the communication side, but does *not* consider the radar function’s object-detection performance at all.

The key criticism driving this paper: all three approaches above use *fixed* schedules for dividing resources between radar and communication. The paper argues this is impractical because the AV’s surrounding environment is *uncertain and dynamic* – weather, traffic, and road conditions all change over time, and a fixed schedule cannot adapt to this.

Illustrative example (from the paper): in bad weather (e.g. heavy rain), the AV should favor radar mode more often to catch unexpected events; in good weather with a good communication channel, it should favor communication mode. Determining the *right, adaptive* mix in real

time, without prior knowledge of how the environment evolves, is the motivating challenge for using reinforcement learning.

1.5 The Paper’s Stated Contribution

- Develops a **Deep Reinforcement Learning (DRL)** technique so the AV can learn optimal radar/communication mode selection *without* requiring prior knowledge of the environment.
- Explicit novelty claim (quoted): “*to the best of our knowledge, this is the first approach using DRL to solve the mode selection problem of the DFRC in AV.*”
- Method: formulate the problem as a **Markov Decision Process (MDP)**, solved using **DQN (Deep Q-Network)** [1].
- Claimed result: the DRL approach outperforms baseline schemes on three metrics – data throughput, miss detection probability, and convergence time.

Remark (beyond the paper’s explicit text)

This is a 4-page IEEE *Letter*, not a full journal article. Section I compresses motivation and contribution into a very tight space with no dedicated related-work section. This is standard for the Letters format, not a shortcoming specific to this paper – but it does mean the treatment of prior work is intentionally brief.

2 System Model

2.1 Operating Principle (Introductory Paragraph)

The AV’s DFRC equipment supports two modes: radar mode and communication mode. The paper makes a deliberate design choice relative to prior work [4]: instead of splitting each time cycle between the two modes, **each entire time step is assigned to exactly one mode** (either radar or communication, not both). This allows the AV to change its mode every single step based on the current environment observation, rather than being locked into a pattern fixed by the previous cycle.

Remark (beyond the paper’s explicit text)

This design choice is what makes the binary action space $a \in \{0, 1\}$ (introduced in Section III) possible. It is also precisely the assumption that rules out any finer-grained resource split – e.g. a partial duty cycle between radar and communication – within a single time step.

2.2 II-A: Dual-Functional Radar-Communication Model

Communication mode. The AV uses V2I capability to transmit data (traffic information, live onboard video streaming) to Base Stations distributed along the road. Modeling assumptions:

- A **single channel** is used for data transmission.
- A **data queue of capacity D** buffers incoming packets (e.g. from the AV’s own sensor devices) before transmission.

Radar mode. The AV performs automotive millimeter-wave radar sensing to detect “unexpected events.”

Definition (unexpected event): an event that is *outside the AV’s camera range* and could cause a collision. The paper’s own example: a car obscured by a truck, emerging from another road. This is specifically the camera-blind-spot case that radar is meant to catch – not simply “any obstacle.”

The four risk factors. The occurrence of an unexpected event is modeled as being influenced by four binary factors:

Factor	State = 1 (unfavorable)	State = 0 (favorable)
Road condition (r)	Slippery road	Straight/dry road
Weather condition (w)	Rainy weather	Good weather
Speed (v)	High speed	Low speed
Moving object (m)	Object nearby	No object nearby

These states are obtained from the AV’s sensing system: road friction sensor, weather station instrument, speedometer, and cameras.

Notation: p_j^i = probability that an unexpected event occurs given factor $i \in \{r, w, v, m\}$ is in condition $j \in \{0, 1\}$. E.g. p_1^r = probability of an unexpected event given a slippery road.

Remark (beyond the paper’s explicit text)

The paper notes that the binary states could be generalized (e.g. speed as low/medium/high), but the actual model implemented in this paper uses only binary states throughout.

2.3 II-B: Environment Model

Data sources for probabilities. p_j^v and p_j^w (speed- and weather-related probabilities) are taken from real-world external data sources: crash risk data [2] and weather-road impact data [3]. The remaining probabilities (p_j^r, p_j^m) are simply assumed/pre-defined by the authors, not sourced from real data.

Combining factors into a single probability. Let τ_i = probability that factor i is in state 0 (favorable), so $1 - \tau_i$ = probability it is in state 1 (unfavorable). The combined probability of an unexpected event, $P(\oplus)$, is given by:

$$P(\oplus) = \sum_{i \in \{r, w, v, m\}} (\tau_i p_0^i + (1 - \tau_i) p_1^i) \quad (1)$$

Remark (beyond the paper’s explicit text)

The paper describes Equation (1) as derived “by using Bayes’ theorem,” but what is actually written is a **sum** of each factor’s own expected contribution across the four factors – not a standard joint-probability computation, which would typically combine (e.g. multiply, under independence) probabilities across factors rather than add them. This is a modeling simplifi-

cation worth keeping in mind if this formulation is revisited later.

Performance metrics defined here.

- **Miss detection probability:** (number of unexpected events the AV fails to detect) / (total unexpected events). Higher values indicate higher accident risk.
- **Data throughput:** average number of packets per time unit successfully transmitted from the AV to the BSs.

Remark (beyond the paper’s explicit text)

Key simplifying assumption: when the AV is in radar mode, detection is assumed *perfect* – no missed detections, no false alarms. The paper explicitly acknowledges this is a simplification and states the model could be extended to account for imperfect radar sensing (miss detection and false alarms caused by sensing accuracy), but this extension is not carried out in the paper itself. This is a *separate* simplification from the “perfect state observation” assumption discussed under Section III – the two should not be conflated, since a research reformulation could address either independently.

The core tradeoff. More radar mode usage $\rightarrow$ lower miss-detection risk but lower throughput. More communication mode usage $\rightarrow$ higher throughput but higher miss-detection risk. This trade-off, under environmental uncertainty, is exactly what gets formalized as an MDP and solved via DRL/DQN in the following sections.

3 Problem Formulation

3.1 The MDP Tuple

The problem is formalized as a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P} \rangle$, where:

- $\mathcal{S}$ = state space
- $\mathcal{A}$ = action space
- $\mathcal{R}$ = reward function
- $\mathcal{P}$ = state transition probability

Remark (beyond the paper’s explicit text)

The paper explicitly states that $\mathcal{P}$ is **unknown** to the AV. This is precisely why DRL – rather than classical dynamic programming, which would require a known transition model – is the appropriate tool: the AV must learn good behavior purely from interaction/experience.

3.2 III-A: Action Space and State Space

Action space.

$$\mathcal{A} = \{a ; a \in \{0, 1\}\}, \quad a = \begin{cases} 0 & \text{communication mode} \\ 1 & \text{radar mode} \end{cases} \quad (2)$$

A binary switch each timestep – nothing more granular. (This is exactly the “continuous action space” limitation the authors flag as future work in their conclusion.)

State space. The AV’s state combines everything from Section II into a 6-tuple:

$$\mathcal{S} = \{(d, c, r, w, v, m) : d \in \{0, 1, \dots, D\}, c \in \{0, 1\}, r, w, v, m \in \{0, 1\}\} \quad (3)$$

where d = data queue state, c = communication channel state, and r, w, v, m are the four risk factors from Section II-A. At timestep t , the AV’s full state is $s_t = (d, c, r, w, v, m) \in \mathcal{S}$ – this is the input the Q-network receives.

Channel state c : $c = 0$ means the channel is *good* (low interference); $c = 1$ means the channel is *bad* (high interference). This refers purely to the communication (V2I) link, not the radar channel.

Remark (beyond the paper’s explicit text)

The paper never specifies how the AV determines c . In the simulation (Section V-A), c is not measured or sensed at all – it is simply drawn randomly each timestep from a fixed probability, $p_c = 0.1$ (10% chance bad, 90% chance good). This is a modeling assumption, not a sensing mechanism. In a real system, c would need to be inferred via SINR/SNR measurement, RSSI, packet error rate / ACK-NACK feedback, a Channel Quality Indicator (CQI), or pilot-signal estimation – none of which give a perfect, instantaneous, noise-free label. This makes c a natural example of a state variable that is, in reality, only *partially observable* – directly relevant to a possible POMDP reformulation of this problem.

3.3 III-B: Reward Function

At each step, the AV selects action a_t at state $s_t \in \mathcal{S}$ and receives reward r_t , defined piecewise as:

$$r_t = \begin{cases} +r_1, & \text{if } a_t = 0, c = 0, \text{ given } \ominus \\ +r_2, & \text{if } a_t = 0, c = 1, \text{ given } \ominus \\ -r_3, & \text{if } a_t = 0, \text{ given } \oplus \\ +r_4(b+1), & \text{if } a_t = 1, \text{ given } \oplus \\ 0, & \text{if } a_t = 1, \text{ given } \ominus \end{cases} \quad (4)$$

where $\oplus$ denotes an unexpected event occurring, $\ominus$ denotes no unexpected event, and b = number of factors currently at state 1 (unfavorable) among $\{r, w, v, m\}$ – so the radar-mode detection reward scales with how risky the conditions were at the moment of a successful detection (recall: radar detection is assumed perfect, per Section II-B).

Remark (beyond the paper’s explicit text)

The comm-mode rewards (r_1, r_2 in Eq. (4)) are triggered simply by being in communication mode with no event occurring – the formula as written does **not** condition this reward on the data queue d actually having packets to send. This is a simplification the paper does not call out explicitly, and it creates a subtle mismatch: the queue dynamics (Section V-A) do depend on d , but the reward function does not reference d at all.

Remark (beyond the paper’s explicit text)

With the experimental weights $(r_1, r_2, r_3, r_4) = (2, 1, 50, 5)$ (given in Section V-A), the penalty for missing an event in comm mode (-50) massively dominates both the small comm-mode rewards ($+1/+2$) and even the maximum radar detection reward ($r_4 \times (b_{\max} + 1) = 5 \times 5 = 25$). This bakes in a strong safety-conservative bias *by construction*, via a single large hand-picked penalty – not via any formal safety guarantee. This is relevant if considering a constrained/safe-RL reformulation, where such guarantees could be made explicit rather than emerging incidentally from reward-weight tuning.

The learning objective. The AV seeks the policy π maximizing expected discounted cumulative reward:

$$\max_{\pi} G(\pi) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_{t+1}(\pi) \right] \quad (5)$$

where $\gamma \in (0, 1)$ is the discount factor and T is the time horizon. The optimal policy π^* gives the optimal action at any state: $a_t^* = \pi^*(s_t)$.

Why DQN, not tabular Q-learning. The paper notes that standard Q-learning (maintaining a table of $Q(s, a)$ values, iteratively updated) suffers from the **large state-action space problem** – with 6 state variables (some higher-cardinality, e.g. $d \in \{0, \dots, D\}$) and 2 actions, a tabular approach scales poorly. This motivates using DQN, where a neural network approximates $Q(s, a)$ instead of a lookup table – detailed next in Section IV.

4 Deep Reinforcement Learning Algorithm

4.1 The Q-Network’s Role

DQN uses a deep neural network, the *Q-network*, with weights θ , to derive an approximate value of $Q^*(s, a)$ – since the true optimal Q-function is unknown (recall $\mathcal{P}$ is unknown to the AV), the network learns to approximate it from experience.

- **Input:** one state of the AV, $s_t = (d, c, r, w, v, m)$.
- **Output:** $Q(s, \cdot, \theta)$ – the Q-values for *all* possible actions simultaneously (both communication and radar mode), not just one.

Remark (beyond the paper’s explicit text)

Architecture details, given later in Section V-A: the Q-network is a simple MLP with 1 input layer, 2 hidden layers, and 1 output layer. The input layer has **6 units** (matching the 6 state dimensions) and the output layer has **2 units** (matching the 2 possible actions).

4.2 The Training Loop

The paper describes the following loop:

1. **At the start of iteration t :** given state $s_t \in \mathcal{S}$, the AV obtains $Q(s, \cdot, \theta)$ for all possible actions.

2. **Action selection:** the AV selects a_t according to an ε -greedy policy [1] – with probability ε , a random action is chosen (exploration); otherwise, the action with the highest Q-value is chosen (exploitation), $a_t = \arg \max_a Q(s_t, a; \theta)$.
3. **Environment interaction:** the AV observes reward r_t and next state s_{t+1} .
4. **Storage:** the transition $m_t = (s_t, a_t, r_t, s_{t+1})$ is stored in a replay memory $\mathcal{M}$.
5. **Learning update:** a random mini-batch of transitions is sampled from $\mathcal{M}$ (not just the most recent transition) to update θ .

Remark (beyond the paper’s explicit text)

Why ε -greedy is needed: the Q-network starts with random, untrained weights, so its earliest Q-value predictions are effectively noise. Without exploration, the AV could get stuck: if the untrained network happens to slightly overestimate $Q(s, \text{comm})$ for some state, a purely greedy policy would always pick communication mode there – and, having never tried radar mode in that state, it would never gather data to correct the (possibly wrong) estimate. ε -greedy guarantees the AV keeps visiting both actions across states during training, so the Q-network’s estimates are learned from real experience rather than reinforcing an early, possibly-incorrect guess. **This mechanism is only active during training** – after training, ε is driven toward zero (standard DQN practice) and mode selection becomes purely $\arg \max_a Q(s, a)$.

4.3 The Weight Update Equation

The paper gives the update rule explicitly:

$$\theta_{t+1} = \theta_t + \alpha [y_t - Q(s_t, a_t; \theta_t)] \nabla Q(s_t, a_t, \theta_t) \quad (6)$$

where:

- α = learning rate
- $\nabla Q(s_t, a_t, \theta_t)$ = gradient of the Q-value with respect to the *online* network’s weights θ_t
- y_t = the target value:

$$y_t = r_t + \gamma \max_a Q(s_{t+1}, a; \theta_t^-) \quad (7)$$

- θ_t^- = **target network weights:** a separate, delayed copy of the network, periodically copied from the online network weights θ_t .

Remark (beyond the paper’s explicit text)

The target network (θ^-) is a standard DQN stabilization technique (not unique to this paper): using the same, constantly-updating network to both select actions and generate its own training targets tends to destabilize learning, so a slower-moving “target” copy is used instead to compute y_t in Eq. (7).

4.4 Convergence Criterion

Steps 1–5 (Section 4.2) repeat across iterations. Training is treated as an **episodic task**, and the algorithm is considered to have converged when the cumulative reward stabilizes across episodes.

Remark (beyond the paper’s explicit text)

What this section does not specify: replay memory size $|\mathcal{M}|$, mini-batch size, and the exact schedule for copying $\theta \rightarrow \theta^-$ (i.e. how many steps/episodes between target network updates) are all left unspecified in the letter. These are standard DQN hyperparameters (from Mnih et al. [1], which the paper cites) that a reader is expected to fill in with typical/default values – consistent with the compressed Letters format noted throughout this document. If re-implementing this baseline, these values must be chosen independently.

Summary of Simplifying Assumptions (Consolidated)

For convenience, here is every simplifying assumption flagged in the Remarks above, in one place – useful as a starting checklist when discussing possible novelty directions:

1. **Fixed per-step mode assignment:** each entire time step is either fully radar or fully communication – no intra-step split (Section 2.1).
2. **Additive, not joint, risk combination:** $P(\oplus)$ is computed as a sum across the four risk factors rather than a true joint probability calculation, despite being described as Bayes’-theorem-based (Section 2.2, Eq. 1).
3. **Perfect radar detection:** no missed detections or false alarms are modeled when the AV is in radar mode (Section 2.2).
4. **Unmeasured channel state:** the good/bad channel state c is not derived from any sensing mechanism – it is simply drawn from a fixed probability each timestep (Section 3.1).
5. **Perfect state observability:** all state variables (d, c, r, w, v, m) are assumed to be known to the AV exactly, with no observation noise – effectively an MDP standing in for what is, in reality, a partially observable problem (Section 3.1).
6. **Reward function ignores queue occupancy:** communication-mode rewards do not check whether the data queue d actually has packets to send (Section 3.2).
7. **Safety via hand-tuned penalty weight, not formal guarantee:** the large miss-detection penalty ($r_3 = 50$) enforces safety-conservative behavior only through manual reward-weight selection, with no probabilistic or worst-case guarantee (Section 3.2).
8. **Binary action space:** the AV can only fully commit to one mode per step – no continuous/partial trade-off between radar and communication is possible (Section 3.1) – explicitly flagged by the authors themselves as future work.
9. **Unspecified DQN hyperparameters:** replay memory size, mini-batch size, and target-network update frequency are left unstated (Section 4).

Sections V (Performance Evaluation) and VI (Conclusion) are intentionally excluded from this document and will be studied separately before finalizing the novelty direction for our own work.

References

- [1] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [2] C. N. Kloeden *et al.*, *Travelling Speed and Risk of Crash Involvement on Rural Roads*, Aust. Transp. Safety Bureau, Canberra, ACT, Australia, 2001.
- [3] *How Do Weather Events Impact Roads*. [Online]. Available: https://ops.fhwa.dot.gov/weather/q1_roadimpact.htm